

GitHub Copilot 入門

AI エージェント編 — 「コード補完」から「仕事を任せる AI」へ

対象: GitHub Copilot 試験導入プロジェクト 立上げチーム／開発部門の新任メンバー

前提資料: 「Git 入門 — 基本概念イントロダクション」

はじめに

GitHub Copilot は、GitHub が提供する AI によるコーディング支援サービスです。当初は「入力中のコードの続きを提案する」補完ツールでしたが、現在は指示を受けて自ら計画し、ファイルを編集し、テストを実行し、プルリクエストまで作成する「AI エージェント」としての機能を備えています。

本資料では、エージェントとしての Copilot の仕組みと、試験導入にあたって組織として決めるべきルールを整理します。AI エージェントは「作業を任せられる」分、任せる範囲・確認の方法・責任の所在をあらかじめ設計しておくことが重要です。これは AI リテラシーや法務の観点が最も活きる領域でもあります。

社内業務での例え	Copilot の機能
文章作成ソフトの予測変換	コード補完 (入力中に続きを提案)
詳しい同僚への質問	Copilot Chat (質問・説明・修正案の提示)
隣の席と一緒に作業するアシスタント	エージェントモード (IDE 内で複数ファイルを自律的に編集・実行)
案件を任せる担当者 (成果物は決裁に回る)	Copilot coding agent (Issue を任せると PR を作成)
起案文書のダブルチェック担当	Copilot code review (PR へのレビューコメント)

1. GitHub Copilot の機能の全体像

Copilot の機能は、人がどこまで主導するかによって段階的に整理できます。下にいくほど AI の自律性が高くなります。

機能	動作場所	概要	人の関与
コード補完	IDE (VS Code、Visual Studio、JetBrains など)	入力中のコードの続きを灰色の文字で提案。Tab キーで採用。	1行ごとに人が採否を判断
Copilot Chat	IDE、github.com、モバイル	自然言語で質問・説明依頼・修正案の作成。コードの解説やテスト作成の相談に。	提案を人が読んで反映
エージェントモード	IDE	目的を伝え、関連ファイルを探して複数ファイルを編集し、ターミナルでコマンドやテストを実行。エラーが出れば自ら修正を繰り返す。	コマンド実行の承認、最終的な変更の確認
Copilot coding agent	GitHub (クラウド)	Issue を Copilot に割り当てると、GitHub Actions 上の隔離環境で作業し、ブランチ・コミット・ドラフト PR を作成。	PR のレビューと承認 (マージ)
Copilot code review	GitHub (PR)	PR の差分を読み、バグの可能性や改善点をレビューコメントとして指摘。	指摘の採否を人が判断
Copilot CLI ほか	ターミナル	コマンドラインからエージェントに作業を依頼。	実行内容の承認

注意: 機能名・提供範囲 (プレビュー／一般提供)・対象プランは頻繁に更新されます。試験導入で使う機能は、GitHub 公式ドキュメントと自社の契約プランで最新状況を必ず確認してください。

2. 「AI アシスタント」と「AI エージェント」の違い

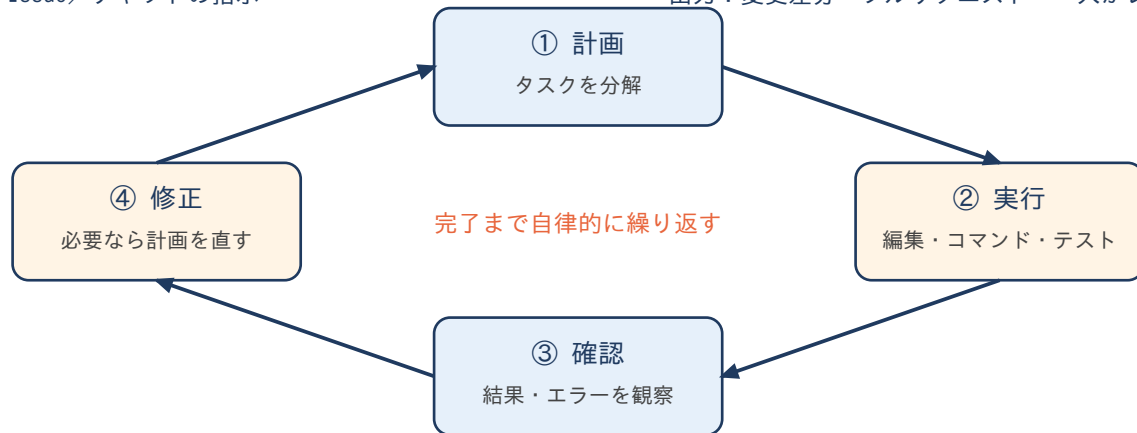
比較項目	AI アシスタント (補完・チャット)	AI エージェント (エージェントモード・coding agent)
動き方	1回の質問に1回答える (一問一答)	目標に向けて、計画 → 実行 → 確認 → 修正を自律的に繰り返す
扱う範囲	主に開いているファイル・選択範囲	リポジトリ全体 (複数ファイルの横断的な変更)
使う道具	なし (テキストを生成するだけ)	ファイル編集、ターミナル実行、テスト、検索、外部ツール (MCP)
成果物	提案 (人がコピー・採用する)	実際の変更差分・コミット・プルリクエスト
人の役割	書き手 (AI は補助)	依頼者・レビュー担当 (AI は作業者)
主なリスク	誤った提案をそのまま採用する	意図しない広範囲の変更、不要なコマンド実行、外部からの不正な指示 (プロンプトインジェクション)
必要な統制	提案内容の確認	権限の範囲設定、実行環境の隔離、PR レビュー必須、ログ・記録の保持

3. AI エージェントの仕組み

AI エージェントは、大規模言語モデル(LLM)に「道具(ツール)」と「繰り返し(ループ)」を組み合わせたものです。LLM が次に何をすべきかを判断し、ファイル編集やコマンド実行などの道具を使い、その結果を見てまた次の行動を決めます。

入力：Issue／チャットの指示

出力：変更差分・プルリクエスト → 人がレビュー

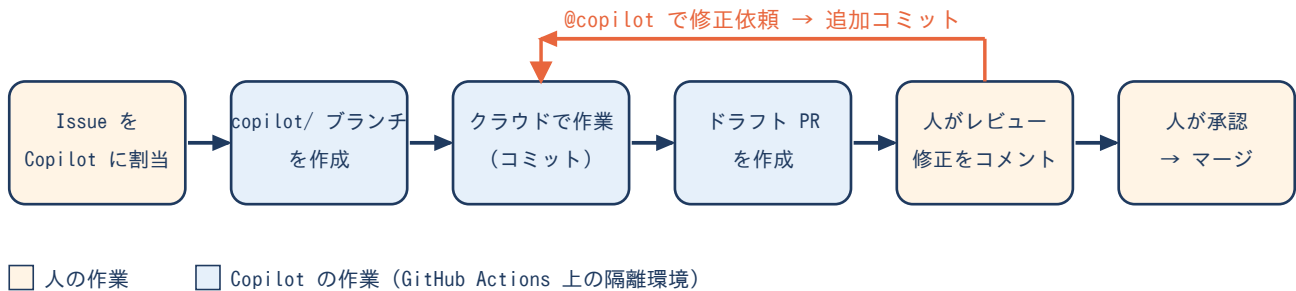


構成要素	説明	Copilot での例
モデル(LLM)	状況を理解し、次の行動を決める「頭脳」	複数の AI モデルから選択可能(プラン・管理者設定による)
コンテキスト	判断材料として与えられる情報	リポジトリのコード、Issue の本文、指示ファイル、会話履歴
ツール	実際に作業を行う手段	ファイルの読み書き、ターミナル、テスト実行、Web 検索、MCP サーバー
指示ファイル	プロジェクト固有のルールを AI に伝える文書	.github/copilot-instructions.md など(コーディング規約・ビルド方法・禁止事項を記載)
MCP	Model Context Protocol。外部ツールやデータを AI に接続する共通規格	課題管理ツール、社内 DB、ドキュメントなどとの連携

ポイント: エージェントの出来は「モデルの賢さ」だけでなく、与える情報(コンテキスト)の質で大きく変わります。Issue に目的・完了条件・制約を明確に書くことは、業務委託で仕様書を明確にすることと同じです。

4. Copilot coding agent と Git／GitHub の関係

Copilot coding agent は、前回の資料で学んだ Git／GitHub の仕組みの上で動きます。AI の作業も通常の開発と同じく「ブランチ → コミット → プルリクエスト → レビュー → マージ」の流れに乗るため、既存のレビュー・承認フローで統制できるのが特長です。



手順	内容	Git／GitHub の概念
1. 依頼	Issue に目的・完了条件を書き、担当者 (Assignee) に Copilot を指定。チャット画面などから依頼することも可能。	Issue
2. 作業	Copilot が専用ブランチを作成し、GitHub Actions 上の隔離された環境でコードを調査・編集・テスト。	ブランチ、Actions
3. 提出	作業内容をコミットし、ドラフトのプルリクエストを作成。進捗やセッションのログを確認できる。	コミット、PR
4. 修正	人が PR にレビューコメント (@copilot を付けて指示) すると、Copilot が追加のコミットで対応。	レビュー
5. 承認	人が内容を確認し、承認・マージ。Copilot 自身はマージできない。	マージ、ブランチ保護

向いているタスクと向いていないタスク

任せやすい (範囲が明確・検証しやすい)	任せにくい (判断・合意が必要)
テストコードの追加、テストカバレッジの改善	要件があいまいな新機能、業務仕様の決定
小さなバグ修正、エラーメッセージの改善	アーキテクチャの大幅な変更
ドキュメント・コメントの整備	セキュリティ・認証・個人情報処理の中核部分
定型的なリファクタリング、ライブラリの更新	複数部署の合意が必要な変更、本番データの操作

5. 安全策とガバナンス

Copilot coding agent には、GitHub が用意している組み込みの安全策があります（主なもの。詳細は公式ドキュメントで確認）。

- 作業できるのは自分が作成したブランチ (copilot/ で始まる名前) のみで、main などへ直接 push できない
- Copilot が作成した PR は人のレビュー・承認が必要。Copilot に作業を依頼した本人は、その PR を承認できない
- Copilot が作成した PR で GitHub Actions のワークフローを実行するには、人の許可が必要
- 作業環境からのインターネットアクセスはファイアウォールで制限できる
- 作業のコミットには依頼者の情報が記録され、誰の依頼による変更かを追跡できる

組織として決めること（管理者設定・運用ルール）

項目	検討内容
機能の有効化範囲	組織・リポジトリ単位で、どの機能（補完・チャット・エージェントモード・coding agent・code review）を誰に許可するか。試験導入は対象リポジトリと対象者を限定するのが安全。
ブランチ保護	main への直接 push 禁止、PR の承認必須、自動テスト合格を必須に設定（前回資料の内容がそのまま統制手段になる）。
コンテンツ除外	機密ファイルを Copilot の参照対象から外す設定。エージェント機能では適用されない場合があるため、対象機能ごとに最新の対応状況を確認する。
パブリックコードとの一致	公開コードと一致する提案をブロックするか、参照情報付きで許可するかを決める。
MCP・外部接続	どの MCP サーバー（外部ツール連携）を許可するか。許可リスト方式を推奨。
シークレット管理	エージェントの作業環境に渡す認証情報は最小限にする。リポジトリへの秘密情報のコミット禁止を徹底。
ログ・監査	監査ログ、エージェントのセッションログ、PR 履歴を保存し、利用状況を定期的に確認する。

6. 法務・リスクの観点からの確認事項

観点	リスク	対応の方向性
責任の所在	AI が作成したコードの不具合の責任が曖昧になる	「マージを承認した人（組織）が責任を負う」原則を明文化。AI は作業者であり、承認者ではない。
著作権・ライセンス	提案コードが既存の公開コードと一致する可能性	パブリックコード一致の抑制設定、OSS ライセンス確認手順、提供元の補償条件（契約）の確認。
機密情報	プロンプトや参照ファイルを通じた機密情報の取扱い	コンテンツ除外、対象リポジトリの限定、入力してはいけない情報のガイドライン化。
プロンプトインジェクション	Issue・Web ページ・外部データに紛れた悪意ある指示に AI が従う	信頼できる人のみが依頼可能にする、外部アクセスの制限、PR レビューでの差分確認。
データの取扱い	入力・出力データの保存期間や学習利用	契約プラン（Business／Enterprise 等）の利用規約・データ保護条項・DPA を確認。
誤り・品質	もっともらしいが誤ったコード（ハルシネーション）	自動テスト必須、人によるレビュー、重要領域は AI 単独作業を禁止。
説明責任・監査	変更理由を後から説明できない	Issue・PR・コミット・セッションログで「誰が・何を依頼し・誰が承認したか」を記録。

ポイント: AI エージェントの統制は、新しい仕組みを一から作るよりも、「依頼（Issue）→ 作業（ブランチ）→ 決裁（PR レビュー）→ 記録（履歴）」という既存の Git／GitHub のフローに乗せることで実現しやすくなります。

7. 試験導入（PoC）の進め方の例

フェーズ	内容	成果物
1. 準備	目的・評価指標の設定、対象リポジトリ・参加者の選定、管理者設定とブランチ保護の確認	導入計画書、利用ガイドライン草案
2. 補助機能から開始	コード補完と Chat を利用し、操作に慣れる	利用者アンケート
3. エージェント機能	テスト追加など任せやすいタスクで、エージェントモード・coding agent を試行	タスクごとの評価記録
4. 評価・判断	効果とリスクを評価し、本格導入の可否と運用ルールを決定	評価報告書、正式ガイドライン

評価指標の例

- 効果: 作業時間の短縮、PR 作成からマージまでの期間、テストカバレッジの変化、利用者の満足度
- 品質: Copilot 作成 PR の差し戻し率・修正回数、マージ後の不具合件数
- 統制: ガイドライン違反の有無、機密情報の混入件数、監査ログの確認結果

8. 用語集

用語	意味
AI エージェント	目標に向けて、計画・ツール実行・確認を自律的に繰り返す AI
LLM	大規模言語モデル。文章やコードを理解・生成する AI の中核
プロンプト	AI への指示文。Issue の本文やチャットの入力など
コンテキスト	AI が判断に使う情報（コード、指示、会話履歴など）
エージェントモード	IDE 内で Copilot が複数ファイルの編集やコマンド実行を自律的に行うモード
Copilot coding agent	Issue を任せるとクラウド上で作業し、PR を作成する Copilot の機能
MCP	Model Context Protocol。AI と外部ツール・データをつなぐ共通規格
カスタム指示	リポジトリ固有のルールを Copilot に伝えるファイル（.github/copilot-instructions.md など）
プロンプトインジェクション	外部データに埋め込まれた悪意ある指示で AI を誤動作させる攻撃
ハルシネーション	AI がもっともらしいが事実と異なる内容を生成すること
Human-in-the-loop	AI の作業の要所に人の確認・承認を組み込む考え方

9. 次のステップ

- 試験導入チームで対象リポジトリ・参加者・評価指標を決める
- 管理者設定（機能の有効化範囲、コンテンツ除外、パブリックコード一致、MCP）とブランチ保護を確認する
- Copilot 利用ガイドライン（任せてよいタスク、入力禁止情報、レビュー責任）を作成する
- テスト用リポジトリで、Issue を Copilot に割り当て → PR レビュー → マージを一通り体験する
- 参考: GitHub Copilot ドキュメント <https://docs.github.com/ja/copilot>

※ 本資料の機能・設定の説明は作成時点の公開情報に基づく概要です。導入判断の際は公式ドキュメントと契約条件の最新版を確認してください。